

SQL - Guía Completa de Instrucciones y Funciones Usadas en CAAJ

Introducción

Este documento explica **TODAS** las instrucciones, funciones y conceptos de SQL que se usan en el proyecto CAAJ. Incluye desde lo más básico como **SELECT** hasta técnicas avanzadas con **INNER JOIN**, **REPLACE**, **UNION**, y más.

Archivos Analizados

- **sql/create.sql** - Define la estructura de las tablas (DDL)
 - **sql/insert.sql** - Inserta datos iniciales (DML)
 - **model.py** - Contiene todas las consultas SQL del backend
-

DDL (Data Definition Language) - Definición de Estructura

CREATE TABLE

¿Qué hace? Crea una nueva tabla en la base de datos.

Sintaxis básica:

```
CREATE TABLE nombre_tabla (  
    columna1 TIPO restricciones,  
    columna2 TIPO restricciones,  
    ...  
);
```

Ejemplo de create.sql:

```
CREATE TABLE `usuario` (  
    `id` INT(11) NOT NULL AUTO_INCREMENT,  
    `apodo` VARCHAR(50) NOT NULL,  
    `email` VARCHAR(100) NOT NULL,  
    `pass` VARCHAR(255) NOT NULL,  
    `id_rol` INT(11) NOT NULL,  
    `nombre` VARCHAR(50) NOT NULL,  
    `apellido` VARCHAR(50) NOT NULL,  
    `dni` VARCHAR(20) NOT NULL,  
    PRIMARY KEY (`id`),  
    UNIQUE KEY `uniq_usuario_email` (`email`),
```

```
UNIQUE KEY `uniq_usuario_dni` (`dni`),  
KEY `fk_usuario_rol` (`id_rol`),  
CONSTRAINT `usuario_ibfk_rol`  
  FOREIGN KEY (`id_rol`)  
  REFERENCES `rol` (`id`)  
  ON DELETE RESTRICT  
  ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

DROP TABLE IF EXISTS

¿Qué hace? Borra una tabla si existe (para poder recrearla limpia).

Ejemplo:

```
DROP TABLE IF EXISTS `usuario`;
```

¿Por qué se usa?

- Evita errores si la tabla ya existe
- Útil para resetear la BD en desarrollo

Tipos de Datos Usados

INT(11)

- **Tipo:** Número entero
- **Uso:** IDs, contadores, referencias
- **Ejemplo:** `id INT(11)`

VARCHAR(n)

- **Tipo:** Texto de longitud variable
- **Uso:** Nombres, emails, apodos
- **Ejemplo:** `apodo VARCHAR(50)` → Máximo 50 caracteres

TEXT

- **Tipo:** Texto largo sin límite fijo
- **Uso:** Contenido de apuntes, comentarios
- **Ejemplo:** `content TEXT`

DATETIME

- **Tipo:** Fecha y hora
- **Formato:** `YYYY-MM-DD HH:MM:SS`
- **Ejemplo:** `'2025-06-10 09:30:00'`

ENUM('opcion1', 'opcion2')

- **Tipo:** Lista cerrada de valores
- **Uso:** Estados, categorías fijas
- **Ejemplo:**

```
`valor` ENUM('like', 'dislike') NOT NULL
```

Solo puede ser 'like' o 'dislike', nada más.

Restricciones (Constraints)

NOT NULL

¿Qué hace? La columna no puede estar vacía.

Ejemplo:

```
`apodo` VARCHAR(50) NOT NULL -- Siempre debe tener un valor
```

AUTO_INCREMENT

¿Qué hace? El valor se auto-genera, empezando en 1 y sumando 1 cada vez.

Ejemplo:

```
`id` INT(11) NOT NULL AUTO_INCREMENT
```

Resultado:

- Primer registro → **id** = 1
- Segundo registro → **id** = 2
- Tercer registro → **id** = 3

PRIMARY KEY

¿Qué hace? Identifica de forma única cada fila.

Características:

- No puede repetirse
- No puede ser NULL
- Solo puede haber una PRIMARY KEY por tabla

Ejemplo:

```
PRIMARY KEY (`id`)
```

UNIQUE KEY

¿Qué hace? Evita valores duplicados en esa columna.

Ejemplo:

```
UNIQUE KEY `uniq_usuario_email` (`email`) -- No puede haber 2 emails iguales
UNIQUE KEY `uniq_reaccion_usuario_apunte` (`id_usuario`,`id_apunte`) --
Combinación única
```

Caso de uso:

- Un usuario solo puede votar UNA vez por apunte
- Dos emails no pueden ser iguales

FOREIGN KEY (Clave Foránea)

¿Qué hace? Conecta dos tablas, asegurando que el valor exista en la tabla referenciada.

Sintaxis:

```
CONSTRAINT nombre_constraint
  FOREIGN KEY (columna_local)
  REFERENCES tabla_externa (columna_externa)
  ON DELETE accion
  ON UPDATE accion
```

Ejemplo:

```
CONSTRAINT `apunte_ibfk_usuario`
  FOREIGN KEY (`id_usuario`)
  REFERENCES `usuario` (`id`)
  ON DELETE CASCADE
  ON UPDATE CASCADE
```

¿Qué significa?

- `id_usuario` en `apunte` debe existir en `usuario.id`
- `ON DELETE CASCADE`: Si borras el usuario, borra sus apuntes
- `ON UPDATE CASCADE`: Si cambias el ID del usuario, actualiza automáticamente

Acciones posibles:

- **CASCADE**: Propaga el cambio (borra/actualiza en cascada)
- **RESTRICT**: Evita borrar si tiene referencias
- **SET NULL**: Pone NULL en la columna

KEY (Índice)

¿Qué hace? Acelera las búsquedas en esa columna.

Ejemplo:

```
KEY `fk_apunte_usuario` (`id_usuario`)
```

¿Cuándo se usa?

- Columnas que se usan en **WHERE**
- Columnas que se usan en **JOIN**
- Columnas con **FOREIGN KEY**

Configuración del Motor

ENGINE=InnoDB

¿Qué es? Motor de almacenamiento de MySQL que soporta:

- Transacciones (COMMIT/ROLLBACK)
- Foreign Keys
- Mejor integridad de datos

CHARSET=utf8mb4

¿Qué es? Codificación que soporta:

- Emojis 🤖
- Tildes (á, é, í, ó, ú)
- Caracteres especiales (ñ, ü)

COLLATE=utf8mb4_general_ci

¿Qué es? Regla de comparación:

- **ci** = Case Insensitive (no distingue mayúsculas/minúsculas)
- **'Juan' = 'juan' = 'JUAN'**

DML (Data Manipulation Language) - Manipulación de Datos

INSERT INTO

¿Qué hace? Inserta datos en una tabla.

Sintaxis 1: Especificando columnas

```
INSERT INTO tabla (columna1, columna2)
VALUES (valor1, valor2);
```

Ejemplo de `insert.sql`:

```
INSERT INTO `rol` (`nombre`)
VALUES
  ('alumno'),
  ('profesor'),
  ('administrador');
```

¿Qué pasa con `id`?

- Como tiene `AUTO_INCREMENT`, se auto-assigna: 1, 2, 3

Sintaxis 2: Múltiples inserciones

```
INSERT INTO `usuario`
  (`apodo`, `email`, `pass`, `id_rol`, `nombre`,
  `apellido`, `dni`)
VALUES
  ('juanp', 'juan.perez@uni.edu', 'SierraLuna123', 1, 'Juan', 'Pérez',
  '12345678'),
  ('martav', 'marta.vega@uni.edu', 'RioAmarillo', 1, 'Marta', 'Vega',
  '87654321'),
  ('profc', 'carlos@uni.edu', 'LlaveMaestra', 2, 'Carlos', 'Gómez',
  '11223344');
```

Ventajas:

- Inserta 3 usuarios en una sola query
- Más eficiente que 3 INSERT separados

Uso en Python (`model.py`)

Ejemplo: Crear usuario

```
def crearUsuario(dic):
    q = """
        INSERT INTO usuario
```

```
        (apodo, email, pass, id_rol, nombre, apellido, dni)
        VALUES (%s, %s, %s, %s, %s, %s, %s)
    """
    val = (
        dic.get('apodo'),
        dic.get('email'),
        dic.get('pass'),
        dic.get('id_rol'),
        dic.get('nombre'),
        dic.get('apellido'),
        dic.get('dni')
    )
    res_insert = insertDB(BASE, q, val)
    return res_insert
```

¿Por qué %s?

- Son **placeholders** para valores seguros
- Previene **SQL Injection**
- Los valores de `val` reemplazan los `%s` de forma segura

SELECT

¿Qué hace? Consulta datos de la base de datos.

Sintaxis básica

```
SELECT columna1, columna2
FROM tabla
WHERE condicion;
```

Ejemplo 1: Todas las columnas

```
SELECT * FROM materia;
```

Resultado:

```
id | materia
---|-----
1  | Matemáticas
2  | Física
3  | Historia
```

Ejemplo 2: Columnas específicas

```
SELECT apodo, email FROM usuario;
```

Resultado:

apodo	email
-----	-----
juanp	juan.perez@uni.edu
martav	marta.vega@uni.edu

Uso en Python (`model.py`)**Ejemplo: Obtener materias**

```
def obtenerMaterias(result):
    q = """ SELECT materia, id from materia
            ORDER BY materia"""
    filas = selectDB(BASE, q)
    result['materias'] = []
    for materia, id in filas or []:
        result['materias'].append({
            'id': id,
            'materia': materia
        })
    return result
```

WHERE

¿Qué hace? Filtra resultados según condiciones.

Operadores

= (Igual)

```
SELECT * FROM usuario WHERE id_rol = 1;
```

→ Solo usuarios con rol alumno

AND (Y lógico)

```
SELECT * FROM usuario WHERE email=%s AND pass=%s;
```


→ Usuario que cumpla AMBAS condiciones (login)

Ejemplo en `model.py`:

```
def obtenerUsuarioXEmailPass(result, email, passw):
    sSql = """SELECT id, apodo, email, pass, id_rol, nombre, apellido, dni
    FROM usuario WHERE email=%s and pass=%s;"""
    val = (email, passw)
    fila = selectDB(BASE, sSql, val)
```

OR (O lógico)

```
SELECT * FROM apunte
WHERE id_materia = 1 OR id_materia = 2;
```

→ Apuntes de Matemáticas O Física

LIKE (Coincidencia parcial)

¿Qué hace? Busca patrones de texto.

Sintaxis:

- `%` = Cualquier cantidad de caracteres
- `_` = Un solo carácter

Ejemplos:

```
-- Buscar apuntes que contengan "límite"
SELECT * FROM apunte WHERE head LIKE '%límite%';
```

Casos:

- 'Límites de funciones' ☒ (contiene "límite")
- 'Derivadas' ☐ (no contiene "límite")

```
-- Buscar emails que terminen en @uni.edu
SELECT * FROM usuario WHERE email LIKE '%@uni.edu';
```

Uso en Python (`model.py`):

```
def buscarGlobal(result, query):
    q = """
```

```
SELECT 'materia' as tipo, m.id, m.materia as titulo
FROM materia m
WHERE LOWER(...) LIKE LOWER(...)
UNION
SELECT 'apunte' as tipo, a.id, a.head as titulo
FROM apunte a
WHERE LOWER(...) LIKE LOWER(...)
LIMIT 10
"""
search_term = f'{query}%' # Agrega % al inicio y final
filas = selectDB(BASE, q, (search_term, search_term, search_term))
```

¿Por qué `f'{query}%'`?

- Usuario busca: `"física"`
- Se convierte en: `"%física%"`
- Encuentra: `"Física"`, `"física cuántica"`, `"Introducción a la física"`

LOWER()

¿Qué hace? Convierte texto a minúsculas.

Ejemplo:

```
SELECT LOWER('Matemáticas');
```

Resultado: `matemáticas`

¿Por qué se usa?

- Búsquedas insensibles a mayúsculas
- Usuario busca `"FISICA"` → encuentra `"Física"`

Uso combinado:

```
WHERE LOWER(a.head) LIKE LOWER('%Límite%')
```

Explicación:

1. `LOWER(a.head)` → `'límites de funciones'`
2. `LOWER('%Límite%')` → `'%límite%'`
3. Compara: `'límites de funciones' LIKE '%límite%'` ☒

REPLACE()

¿Qué hace? Reemplaza texto dentro de un string.

Sintaxis:

```
REPLACE(texto, 'buscar', 'reemplazar')
```

Ejemplo básico:

```
SELECT REPLACE('Matemáticas', 'á', 'a');
```

Resultado: Matemáticas

Normalización de Tildes

Problema: MySQL compara con tildes:

- 'Física' ≠ 'Fisica'

Solución: Quitar tildes de ambos lados:

```
WHERE LOWER(
    REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
        m.materia, 'á', 'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u')
) LIKE LOWER(
    REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
        %s, 'á', 'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u')
)
```

Flujo:

1. Usuario busca: "fisica"
2. BD tiene: "Física"
3. REPLACE(...) quita tildes: "Fisica"
4. LOWER(...) convierte: "fisica"
5. Comparación: "fisica" LIKE "fisica" ☒

Ejemplo completo en model.py:

```
def buscarGlobal(result, query):
    # buscamos en toda la BD con normalización de tildes para buscar mejor
    q = """
        SELECT 'materia' as tipo, m.id, m.materia as titulo, NULL as contenido,
        m.id as materia_id
        FROM materia m
        WHERE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(m.materia, 'á', 'a'),
        'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
        LIKE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(%s, 'á', 'a'), 'é',
        'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
    """
```

UNION

```

SELECT 'apunte' as tipo, a.id, a.head as titulo, a.content as contenido,
a.id_materia as materia_id
FROM apunte a
WHERE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(a.head, 'á', 'a'),
'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
LIKE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(%s, 'á', 'a'), 'é',
'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
OR LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(a.tags, 'á', 'a'), 'é',
'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
LIKE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(%s, 'á', 'a'), 'é',
'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))

LIMIT 10
"""

```

JOIN - Unir Tablas

INNER JOIN

¿Qué hace? Combina filas de dos tablas cuando hay una coincidencia.

Sintaxis:

```

SELECT columnas
FROM tabla1
INNER JOIN tabla2 ON tabla1.columna = tabla2.columna;

```

Diagrama:

Tabla A	Tabla B	
-----	-----	
1	1	← Match ☒
2	3	
3		← Match ☒
4	5	

INNER JOIN → Solo devuelve filas que coincidan en ambas tablas (1, 3)

Ejemplo de `model.py`:

```

SELECT a.id, a.head, a.content, a.tags, u.apodo, m.materia
FROM apunte AS a
INNER JOIN usuario AS u ON a.id_usuario = u.id

```

```
INNER JOIN materia AS m ON m.id = a.id_materia
WHERE m.id = %s
```

¿Qué hace?

1. Toma la tabla **apunte** (alias **a**)
2. Une con **usuario** (alias **u**) donde **a.id_usuario = u.id**
3. Une con **materia** (alias **m**) donde **m.id = a.id_materia**
4. Filtra por materia específica

Resultado:

id	head	content	tags	apodo	materia
1	Límites...	Definición	cálculo, ...	juanp	Matemáticas

Sin JOIN, necesitarías 3 queries:

```
SELECT * FROM apunte WHERE id=1;           -- Obtienes id_usuario=1,
id_materia=1
SELECT apodo FROM usuario WHERE id=1;       -- Obtienes 'juanp'
SELECT materia FROM materia WHERE id=1;    -- Obtienes 'Matemáticas'
```

Con JOIN → 1 sola query 🚀

LEFT JOIN

¿Qué hace? Trae TODAS las filas de la tabla izquierda, aunque no haya coincidencia.

Diagrama:

Tabla A (LEFT)	Tabla B	
1	1	← Match ☒
2		← Sin match, pero se incluye con NULL
3	3	← Match ☒

LEFT JOIN → Devuelve 1, 2 (con NULL en columnas de B), 3

Ejemplo de **model.py**:

```
SELECT a.id, a.id_usuario, a.id_materia, a.head, a.content, a.tags, a.fechahora,
m.materia
FROM apunte a
LEFT JOIN materia m ON a.id_materia = m.id
```

```
WHERE a.id_usuario=%s
ORDER BY a.fechahora DESC;
```

¿Por qué LEFT JOIN?

- Si un apunte tiene `id_materia` inválido (por error), igual lo trae
- `m.materia` será `NULL` en ese caso
- `INNER JOIN` lo habría omitido

Caso de uso:

```
apunte.id_materia = 999 (no existe en tabla materia)
```

```
LEFT JOIN → Trae el apunte con materia = NULL
```

```
INNER JOIN → No lo trae (requiere coincidencia)
```

AS - Alias

¿Qué hace? Le pone un nombre temporal a columnas o tablas.

Alias de columnas

Ejemplo:

```
SELECT 'materia' as tipo, m.id, m.materia as titulo
FROM materia m
```

Resultado:

tipo	id	titulo
materia	1	Matemáticas
materia	2	Física

¿Por qué usar alias?

- Columnas con nombres más claros
- Columnas calculadas necesitan nombre
- JavaScript puede leer `data.tipo` en vez de `data['materia']`

Alias de tablas

Ejemplo:

```
FROM apunte AS a
INNER JOIN usuario AS u ON a.id_usuario = u.id
```

Ventajas:

- Escribe `a.id` en vez de `apunte.id`
- Más legible en queries complejos
- Necesario cuando haces JOIN con la misma tabla 2 veces

UNION

¿Qué hace? Combina resultados de múltiples `SELECT` en una sola tabla.

Reglas:

- Mismo número de columnas
- Tipos de datos compatibles
- Elimina duplicados automáticamente (usa `UNION ALL` para mantenerlos)

Ejemplo de `model.py`:

```
SELECT 'materia' as tipo, m.id, m.materia as titulo
FROM materia m
WHERE LOWER(...) LIKE LOWER(...)

UNION

SELECT 'apunte' as tipo, a.id, a.head as titulo
FROM apunte a
WHERE LOWER(...) LIKE LOWER(...)

LIMIT 10
```

Resultado combinado:

tipo	id	titulo
materia	2	Física
apunte	1	Límites de funciones
apunte	5	Física cuántica

¿Por qué UNION?

- Buscamos en DOS tablas distintas (materia y apunte)
- Queremos UN solo resultado ordenado
- La columna `tipo` nos dice de dónde vino cada fila

Sin UNION necesitarías:

1. Query para materias → Array 1
 2. Query para apuntes → Array 2
 3. Combinar en Python → Más lento
-

ORDER BY

¿Qué hace? Ordena los resultados.

Sintaxis:

```
ORDER BY columna [ASC|DESC]
```

- **ASC** = Ascendente (A→Z, 1→9, fecha antigua→reciente) **[por defecto]**
- **DESC** = Descendente (Z→A, 9→1, fecha reciente→antigua)

Ejemplos:

```
-- Apuntes más recientes primero
SELECT * FROM apunte ORDER BY fechahora DESC;
```

```
-- Materias en orden alfabético
SELECT * FROM materia ORDER BY materia ASC;
```

Uso en model.py:

```
def obtenerApuntesXUsuario(result, id_usuario):
    q = """SELECT a.id, a.id_usuario, a.id_materia, a.head, a.content, a.tags,
a.fechahora, m.materia
        FROM apunte a
        LEFT JOIN materia m ON a.id_materia = m.id
        WHERE a.id_usuario=%s
        ORDER BY a.fechahora DESC;"""
```

Resultado: Apuntes del más nuevo al más viejo.

GROUP BY

¿Qué hace? Agrupa filas que tienen valores iguales en una columna.

Se usa con funciones de agregación:

- `COUNT()` - Contar
- `SUM()` - Sumar
- `AVG()` - Promedio
- `MAX()` - Máximo
- `MIN()` - Mínimo

Ejemplo de `model.py`:

```
q_count = "SELECT valor, COUNT(*) FROM reaccion WHERE id_apunte=%s GROUP BY valor"
```

¿Qué hace?

1. Filtra por apunte específico
2. Agrupa por tipo de voto ('like' o 'dislike')
3. Cuenta cuántos hay de cada tipo

Datos de ejemplo:

id_apunte	valor
1	like
1	like
1	dislike
1	like

Resultado con GROUP BY:

valor	COUNT(*)
like	3
dislike	1

Sin GROUP BY:

```
SELECT COUNT(*) FROM reaccion WHERE id_apunte=1;
```

Resultado: 4 (total, pero no separado)

Uso completo en código:

```
def votar_apunte_db(id_apunte, id_usuario, tipo):  
    # ... lógica de insertar/borrar voto ...
```

```
# Contar votos por tipo
q_count = "SELECT valor, COUNT(*) FROM reaccion WHERE id_apunte=%s GROUP BY
valor"
filas = selectDB(BASE, q_count, (id_apunte,))

likes = dislikes = 0
for tipo_v, count in filas or []:
    if tipo_v == 'like':
        likes = count
    elif tipo_v == 'dislike':
        dislikes = count

return True, likes, dislikes, voto_actual
```

COUNT()

¿Qué hace? Cuenta el número de filas.

Sintaxis:

```
COUNT(*)          -- Cuenta todas las filas
COUNT(columna)   -- Cuenta filas donde columna NO es NULL
COUNT(DISTINCT columna) -- Cuenta valores únicos
```

Ejemplos:

```
-- Cuántos apuntes hay en total
SELECT COUNT(*) FROM apunte;
```

```
-- Cuántos comentarios tiene un apunte
SELECT COUNT(*) FROM comentario WHERE id_apunte=1;
```

```
-- Cuántos usuarios son alumnos
SELECT COUNT(*) FROM usuario WHERE id_rol=1;
```

LIMIT

¿Qué hace? Limita el número de resultados.

Sintaxis:

```
LIMIT n          -- Devuelve solo las primeras n filas
LIMIT offset, n -- Salta 'offset' filas y devuelve n
```

Ejemplo de `model.py`:

```
SELECT 'materia' as tipo, m.id, m.materia as titulo
FROM materia m
WHERE LOWER(...) LIKE LOWER(...)
UNION
SELECT 'apunte' as tipo, a.id, a.head as titulo
FROM apunte a
WHERE LOWER(...) LIKE LOWER(...)
LIMIT 10
```

¿Por qué LIMIT 10?

- Evita devolver 1000 resultados en una búsqueda
- Mejora la performance
- UX: Solo muestra los 10 más relevantes

Ejemplo de paginación:

```
-- Página 1: primeros 10 resultados
SELECT * FROM apunte ORDER BY fechahora DESC LIMIT 10;

-- Página 2: siguientes 10 resultados
SELECT * FROM apunte ORDER BY fechahora DESC LIMIT 10, 10;

-- Página 3: siguientes 10 resultados
SELECT * FROM apunte ORDER BY fechahora DESC LIMIT 20, 10;
```

UPDATE

¿Qué hace? Modifica datos existentes.

Sintaxis:

```
UPDATE tabla
SET columna1 = valor1, columna2 = valor2
WHERE condicion;
```

⚠ **IMPORTANTE:** Siempre usar `WHERE`, sino actualiza TODAS las filas.

Ejemplos:

```
-- Cambiar el apodo de un usuario
UPDATE usuario
SET apodo = 'nuevo_apodo'
WHERE id = 5;
```

```
-- Cambiar rol de usuario a profesor
UPDATE usuario
SET id_rol = 2
WHERE email = 'juan.perez@uni.edu';
```

Uso en Python (`model.py` - ejemplo hipotético):

```
def actualizarApunte(id_apunte, titulo, contenido):
    q = """
        UPDATE apunte
        SET head = %s, content = %s
        WHERE id = %s
    """
    val = (titulo, contenido, id_apunte)
    updateDB(BASE, q, val)
```

DELETE

¿Qué hace? Borra filas de una tabla.

Sintaxis:

```
DELETE FROM tabla
WHERE condicion;
```

⚠ **PELIGRO:** Sin `WHERE` borra TODA la tabla.

Ejemplo de `model.py`:

```
def votar_apunte_db(id_apunte, id_usuario, tipo):
    # Si vota lo mismo, quitamos el voto (toggle)
    if voto_existente == tipo:
        q_del = "DELETE FROM reaccion WHERE id_apunte=%s AND id_usuario=%s"
        deleteDB(BASE, q_del, (id_apunte, id_usuario))
```

Otro ejemplo:

```
-- Borrar un comentario específico
DELETE FROM comentario WHERE id = 42;
```

Con CASCADE:

Cuando defines **ON DELETE CASCADE** en la FOREIGN KEY:

```
CONSTRAINT `comentario_ibfk_apunte`
  FOREIGN KEY (`id_apunte`)
  REFERENCES `apunte` (`id`)
  ON DELETE CASCADE
```

Si haces:

```
DELETE FROM apunte WHERE id = 1;
```

Se borran automáticamente:

- Todos los comentarios de ese apunte
- Todos los votos de ese apunte
- Todos los archivos de ese apunte

Funciones Especiales de SQL

NOW()

¿Qué hace? Devuelve la fecha y hora actual.

Ejemplo en **insert.sql**:

```
INSERT INTO comentario (id_apunte, id_usuario, content, fechahora)
VALUES (%s, %s, %s, NOW())
```

Resultado: '2025-12-05 14:30:45'

NULL

¿Qué es? Ausencia de valor (diferente de 0 o '').

Ejemplo:

```
SELECT 'materia' as tipo, m.id, m.materia as titulo, NULL as contenido
```

¿Por qué NULL?

- Las materias no tienen contenido
- Pero necesitamos esa columna para el UNION con apuntes

Comprobar NULL:

```
WHERE columna IS NULL  
WHERE columna IS NOT NULL
```

✗ INCORRECTO:

```
WHERE columna = NULL -- Siempre es FALSE
```

DISTINCT

¿Qué hace? Elimina duplicados.

Ejemplo:

```
-- Ver qué materias tienen apuntes (sin repetir)  
SELECT DISTINCT id_materia FROM apunte;
```

Sin DISTINCT:

```
id_materia  
-----  
1  
1  
2  
1  
3
```

Con DISTINCT:

```
id_materia  
-----  
1
```

```
2  
3
```

Seguridad: Prevención de SQL Injection

VULNERABLE

```
# ¡NUNCA HAGAS ESTO!  
query = f"SELECT * FROM usuario WHERE email='{email}'"  
cursor.execute(query)
```

Ataque:

```
email = "'; DROP TABLE usuario; --"
```

Query resultante:

```
SELECT * FROM usuario WHERE email=''; DROP TABLE usuario; --'
```

Resultado: ¡Borra toda la tabla! 💀

☒ SEGURO (Prepared Statements)

```
q = "SELECT * FROM usuario WHERE email=%s AND pass=%s"  
val = (email, passw)  
cursor.execute(q, val)
```

¿Qué hace?

1. MySQL trata **%s** como **valores**, no como código SQL
2. Escapa caracteres especiales automáticamente
3. Previene inyección de código malicioso

Ejemplo de ataque bloqueado:

```
email = "'; DROP TABLE usuario; --"  
passw = "password"
```

MySQL interpreta:

```
SELECT * FROM usuario WHERE email=''; DROP TABLE usuario; --' AND pass='password'
```

Como **texto literal**, no como comandos.

Queries Completas del Proyecto

1. Búsqueda Global con Normalización

```
def buscarGlobal(result, query):
    q = """
        SELECT 'materia' as tipo, m.id, m.materia as titulo, NULL as contenido,
        m.id as materia_id
        FROM materia m
        WHERE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(m.materia, 'á', 'a'),
        'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
        LIKE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(%s, 'á', 'a'), 'é',
        'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))

        UNION

        SELECT 'apunte' as tipo, a.id, a.head as titulo, a.content as contenido,
        a.id_materia as materia_id
        FROM apunte a
        WHERE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(a.head, 'á', 'a'),
        'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
        LIKE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(%s, 'á', 'a'), 'é',
        'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
        OR LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(a.tags, 'á', 'a'), 'é',
        'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
        LIKE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(%s, 'á', 'a'), 'é',
        'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))

        LIMIT 10
    """
    search_term = f'%{query}%'
    filas = selectDB(BASE, q, (search_term, search_term, search_term))
```

Técnicas usadas:

- ☒ **LOWER()** - Búsqueda insensible a mayúsculas
- ☒ **REPLACE()** - Normalización de tildes (5 veces encadenado)
- ☒ **LIKE** con **%** - Coincidencia parcial
- ☒ **UNION** - Combinar resultados de materias y apuntes
- ☒ **LIMIT 10** - Solo 10 resultados

- ☒ **AS tipo** - Distinguir origen de cada resultado

2. Apuntes de Materia con Búsqueda

```
def obtenerApuntesPorMateria(result, id_materia, search):
    if search:
        q = """SELECT a.id, a.head, a.content, a.tags, u.apodo, m.materia
        FROM apunte AS a
        INNER JOIN usuario AS u ON a.id_usuario = u.id
        INNER JOIN materia AS m ON m.id = a.id_materia
        WHERE m.id=%s
        AND (
            LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(a.head, 'á',
'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
            LIKE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(%s, 'á',
'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
            OR
            LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(a.tags, 'á',
'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
            LIKE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(%s, 'á',
'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
            OR
            LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(u.apodo, 'á',
'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
            LIKE LOWER(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(%s, 'á',
'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u'))
        )"""
        search_term = f'%{search}%'
        val = (id_materia, search_term, search_term, search_term)
```

Técnicas usadas:

- ☒ **INNER JOIN** (2 veces) - Trae datos de usuario y materia
- ☒ **WHERE m.id=%s** - Filtra por materia específica
- ☒ **AND (... OR ... OR ...)** - Busca en título, tags o usuario
- ☒ Normalización completa en 3 campos

3. Contar Votos

```
def votar_apunte_db(id_apunte, id_usuario, tipo):
    # ... lógica de votar ...

    q_count = "SELECT valor, COUNT(*) FROM reaccion WHERE id_apunte=%s GROUP BY
valor"
    filas = selectDB(BASE, q_count, (id_apunte,))

    likes = dislikes = 0
    for tipo_v, count in filas or []:
```

```

        if tipo_v == 'like':
            likes = count
        elif tipo_v == 'dislike':
            dislikes = count

    return True, likes, dislikes, voto_actual

```

Técnicas usadas:

- ☒ **COUNT(*)** - Cuenta votos
- ☒ **GROUP BY valor** - Separa likes de dislikes
- ☒ Loop en Python para asignar valores

4. Comentarios de Apunte

```

def obtenerComentariosDB(result, id_apunte):
    q = """
        SELECT c.id, c.id_usuario, u.apodo, c.content, c.fechahora
        FROM comentario AS c
        INNER JOIN usuario u ON c.id_usuario = u.id
        WHERE c.id_apunte=%s
        ORDER BY c.fechahora ASC
    """
    val = (id_apunte,)
    filas = selectDB(BASE, q, val)

```

Técnicas usadas:

- ☒ **INNER JOIN** - Trae nombre de usuario
- ☒ **WHERE** - Filtra por apunte
- ☒ **ORDER BY fechahora ASC** - Más viejos primero



Resumen de Instrucciones por Categoría

DDL (Definición de Estructura)

Instrucción	Función
CREATE TABLE	Crear tabla nueva
DROP TABLE	Borrar tabla
PRIMARY KEY	Identifica filas únicas
FOREIGN KEY	Conecta tablas
UNIQUE KEY	Evita duplicados

Instrucción	Función
<code>AUTO_INCREMENT</code>	Auto-numerar IDs
<code>NOT NULL</code>	Obligatorio
<code>ENUM()</code>	Lista cerrada de valores

DML (Manipulación de Datos)

Instrucción	Función
<code>SELECT</code>	Consultar datos
<code>INSERT INTO</code>	Insertar datos
<code>UPDATE</code>	Modificar datos
<code>DELETE</code>	Borrar datos

Filtros y Condiciones

Instrucción	Función
<code>WHERE</code>	Filtrar resultados
<code>AND</code>	Ambas condiciones
<code>OR</code>	Cualquiera de las condiciones
<code>LIKE</code>	Coincidencia parcial
<code>IN (...)</code>	Dentro de una lista
<code>IS NULL</code>	Comprobar valor vacío

Funciones de Texto

Instrucción	Función
<code>LOWER()</code>	Convertir a minúsculas
<code>UPPER()</code>	Convertir a mayúsculas
<code>REPLACE()</code>	Reemplazar texto
<code>CONCAT()</code>	Unir strings

Funciones de Agregación

Instrucción	Función
<code>COUNT()</code>	Contar filas
<code>SUM()</code>	Sumar valores

Instrucción	Función
AVG()	Promedio
MAX()	Valor máximo
MIN()	Valor mínimo

Unión de Tablas

Instrucción	Función
INNER JOIN	Solo coincidencias
LEFT JOIN	Todas de la izquierda + coincidencias
RIGHT JOIN	Todas de la derecha + coincidencias

Organización de Resultados

Instrucción	Función
ORDER BY	Ordenar
GROUP BY	Agrupar
LIMIT	Limitar cantidad
UNION	Combinar queries
DISTINCT	Eliminar duplicados

Alias y Nombres

Instrucción	Función
AS	Renombrar columnas/tablas

Casos de Uso Reales del Proyecto

1. Login de Usuario

```
SELECT id, apodo, email, pass, id_rol, nombre, apellido, dni
FROM usuario
WHERE email=%s AND pass=%s;
```

Técnicas: SELECT, WHERE, AND

2. Ver Apuntes de un Usuario

```
SELECT a.id, a.id_usuario, a.id_materia, a.head, a.content, a.tags, a.fechahora,
m.materia
FROM apunte a
LEFT JOIN materia m ON a.id_materia = m.id
WHERE a.id_usuario=%s
ORDER BY a.fechahora DESC;
```

Técnicas: SELECT, LEFT JOIN, WHERE, ORDER BY DESC

3. Buscar "Física"

```
SELECT 'materia' as tipo, m.id, m.materia as titulo
FROM materia m
WHERE LOWER(REPLACE(..., m.materia, ...)) LIKE LOWER(REPLACE(..., %s, ...))
UNION
SELECT 'apunte' as tipo, a.id, a.head as titulo
FROM apunte a
WHERE LOWER(REPLACE(..., a.head, ...)) LIKE LOWER(REPLACE(..., %s, ...))
LIMIT 10
```

Técnicas: SELECT, AS, LOWER(), REPLACE(), LIKE, UNION, LIMIT

4. Votar un Apunte

```
-- Insertar voto
INSERT INTO reaccion (id_apunte, id_usuario, valor) VALUES (%s, %s, %s);

-- Contar votos
SELECT valor, COUNT(*) FROM reaccion WHERE id_apunte=%s GROUP BY valor;
```

Técnicas: INSERT, SELECT, COUNT(), WHERE, GROUP BY

5. Crear Apunte con Fecha Actual

```
INSERT INTO apunte (id_usuario, id_materia, head, content, tags, fechahora)
VALUES (%s, %s, %s, %s, %s, NOW())
```

Técnicas: INSERT, NOW()

¡Fin de la guía de SQL! 🐼

Ahora entendés cada instrucción SQL que usa el proyecto CAAJ, desde **CREATE TABLE** hasta queries complejos con **INNER JOIN** y normalización de tildes.